SPRINT 03 · ANGULAR · LAB02

Routing

Rutas, navegación, parámetros, carga perezosa, precarga y guards

Ejercicios de entrenamiento. Hay tres niveles y cada uno es un proyecto distinto e independiente, así que no necesitas el anterior para empezar. Los enunciados no incluyen la solución: usa «Cómo saber que está bien» para comprobar tu trabajo.

Nivel	Proyecto	Practicas	Tiempo orientativo
Fácil	Portfolio personal	Definición de rutas, `router-outlet`, `routerLink`, `routerLinkActive` y página 404	45-60 min
Medio	Tienda de discos	Parámetros de ruta, query params, rutas hijas, navegación por código y rutas relativas	2-2,5 h
Difícil	Intranet de clínica	Carga perezosa, guards funcionales, resolvers, `returnUrl` y precarga selectiva	4-5 h

> Trabaja con Angular 22, componentes standalone, `@if` / `@for` e `inject()`. Crea cada proyecto por separado con `ng new <nombre> --style=scss --ssr=false`. Si prefieres decoradores (`@Input`, `@Output`) o funciones (`input()`, `output()`), ambas opciones valen. Angular 22 no usa Zone.js: guarda en un `signal` el estado que cambie fuera de un evento de plantilla (temporizadores, HTTP o suscripciones al router), o la vista no se actualizará.

Fácil

45-60 min

`portfolio`

Portfolio personal

Practicas: Definición de rutas, `router-outlet`, `routerLink`, `routerLinkActive` y página 404

Vas a montar un portfolio con varias páginas sin recargar el navegador. Primero corrige una configuración de rutas defectuosa.

Encuentra los cinco errores

Explica qué provocaría cada error y escribe la versión corregida:

```
export const routes: Routes = [
  { path: '**', component: NoEncontrada },
  { path: '', redirectTo: 'inicio' },
  { path: '/proyectos', component: Proyectos },
  { path: 'proyectos/id', component: ProyectoDetalle },
  { path: 'contacto', Component: Contacto },
];
```

Requisitos

1. Crea las páginas `Inicio`, `Proyectos`, `ProyectoDetalle`, `Contacto` y `NoEncontrada`.
2. Configura las rutas corregidas con `provideRouter` y coloca `<router-outlet />` entre una cabecera y un pie fijos.
3. Añade un menú con `routerLink`. Marca el enlace activo con `routerLinkActive`. «Inicio» no debe quedar activo en otras páginas.
4. En `Proyectos`, enlaza cada proyecto con su detalle (`/proyectos/2`). En el detalle, lee y muestra el `id` con `ActivatedRoute`.
5. Añade la propiedad `title` a cada ruta para cambiar el título de la pestaña.

Cómo saber que está bien

- Al navegar, la cabecera no parpadea y la pestaña Red no recarga el documento.
- Entrar en `/` te lleva a `/inicio`, y una URL inventada muestra la página 404.
- Solo hay un enlace activo en cada página.

Preguntas

- ¿Por qué importa el orden de las rutas? ¿Qué hace `pathMatch: 'full'`?
- ¿Qué diferencia hay entre usar `href` y `routerLink` en una SPA?

Medio

2-2,5 h

`tienda-discos`

Tienda de discos

Practicas: Parámetros de ruta, query params, rutas hijas, navegación por código y rutas relativas

En una tienda de discos, los filtros del catálogo se guardan en la URL para poder compartirlos. Cada disco tiene su ficha, con pestañas propias.

Modelo

```
export interface Disco {
  id: number;
  titulo: string;
  artista: string;
  genero: 'rock' | 'jazz' | 'pop';
  precio: number;
  canciones: string[];
}
```

Rutas que debes construir

Ruta	Contenido
/discos	Catálogo con filtros: ?genero=rock&orden=precio
/discos/:id	Ficha del disco, con un router-outlet propio
/discos/:id/canciones	Ruta hija: lista de canciones (pestaña por defecto)
/discos/:id/resenas	Ruta hija: reseñas

Requisitos

1. Crea un `DiscosService` con ocho discos en memoria y el método `porId(id)`.
2. En el catálogo, los enlaces de género usan `queryParams`. Cambiar el género debe conservar el orden: usa `queryParamsHandling: 'merge'`.
3. El catálogo lee los filtros de la URL. Al recargar la página, los filtros se mantienen.
4. En la ficha, añade «Disco siguiente» y «Volver al catálogo» con `Router.navigate`. «Volver» usa una ruta **relativa** con `relativeTo`.
5. Comprueba que «Disco siguiente» actualiza los datos. Lee el `id` mediante `paramMap`, no con `snapshot`.
6. Si el disco no existe, redirige a la página 404 usando `navigate` con `skipLocationChange: true`.
7. Entrar en `/discos/3` debe llevar a la pestaña de canciones.

Cómo saber que está bien

- Copiar `/discos?genero=jazz&orden=precio` en otra pestaña muestra exactamente el mismo catálogo.
- Al pulsar «Disco siguiente», el título cambia aunque Angular reutilice el componente.
- Tras una redirección a la 404, la barra de direcciones conserva la URL que escribiste.

Preguntas

- ¿Por qué `snapshot` falla con «Disco siguiente»? ¿Cuándo sí es correcto usarlo?
- ¿Cuándo pondrías un dato en la ruta (`:id`) y cuándo en un query param?

Difícil

4-5 h

`intranet-clinica`

Intranet de clínica

Practicas: Carga perezosa, guards funcionales, resolvers, `returnUrl` y precarga selectiva

La intranet de una clínica tiene zonas según el rol del usuario. Se usa `AuthService` simulado: no hay servidor, los datos son ficticios y el login solo elige un rol.

Acceso por rol

Ruta	Rol	Carga	Precarga
/login	Público	Inicial	-
/citas	recepcion, medico	Perezosa	Sí
/pacientes	medico	Perezosa	Sí
/pacientes/:id/editar	medico	Dentro de pacientes	-
/admin	admin	Perezosa	No

Requisitos

1. Carga cada zona con `loadChildren` y su propio archivo de rutas, como `pacientes.routes.ts`.
2. Crea `rolGuard` (`CanActivateFn`), que lee los roles desde `route.data`. Si no hay sesión, redirige a `/login?returnUrl=...`. Si hay sesión pero no permiso, redirige a `/sin-permiso`. Devuelve un `UrlTree`.
3. Protege todas las rutas hijas de `/admin` con un `CanActivateChildFn`.
4. Tras el login, vuelve a la `returnUrl`, o a `/citas` si no existe. Acepta solo rutas internas que empiecen por una única barra `/`.
5. En `pacientes/:id/editar`, usa un `ResolveFn` que entregue el paciente tras simular 500 ms de espera. El componente lo lee de `data`. Si no existe, redirige a `/pacientes`.
6. Crea un `CanDeactivateFn<ConCambios>` genérico que pida confirmación si hay cambios sin guardar. Define `interface ConCambios { tieneCambios(): boolean; }`.
7. Crea una estrategia `PreloadingStrategy` que precargue solo las rutas con `data: { preload: true }`, y actívala con `withPreloading`.

Cómo saber que está bien

- Un usuario sin sesión que entra en `/pacientes/2/editar` vuelve a esa URL después del login.
- Una `returnUrl` de otro dominio, como `https://ejemplo.com` o `//ejemplo.com`, se ignora.
- En la pestaña Red, los bloques de citas y pacientes se descargan solos; el de admin no se descarga hasta entrar.
- Salir de un formulario modificado pide confirmación; salir sin cambios, no.

Preguntas

- Un guard no protege los datos. ¿Por qué? ¿Qué debe validar el servidor en una aplicación real?
- ¿Por qué no conviene precargar `/admin`? ¿Qué aportaría `canMatch` frente a `canActivate` en esa ruta?
- ¿Qué ve el usuario mientras se resuelve el `ResolveFn`? ¿Cómo mostrarías un indicador de carga?